Runnables in LangChain

CampusX · Generative AI using LangChain · Videos 8 & 9 05 / 11 / 2026

Video 8 — What are Runnables & WHY do we need them?

1. The background story

- LLM apps exploded after ChatGPT → OpenAI & others gave **APIs**, so anyone could build on top of LLMs.
- But a real LLM app needs **many parts**: document loader, text splitter, embeddings, vector store, retriever, LLM, output parser...
- LangChain's goal: give a ready-made component for each part so building apps becomes easy.

2. Then came "Chains"

- Team noticed some patterns repeat in almost every app, e.g. **prompt → LLM**.
- So they made a chain for it: **LLMChain**. Then more: **RetrievalQAChain**, **SequentialChain**, **SimpleSequentialChain**, ...
- Every new use case = a new chain class.

3. The problem (why it broke)

- Too many chains → codebase became huge & heavy.
- Steep learning curve → new devs didn't know which chain to use when.

> **ROOT CAUSE** → components were **not standardized**. Each one had its own method name, so they couldn't plug into each other automatically. Every combo needed custom "glue" code = a new chain.

Component	Old method (all different!)
LLM	.predict()
PromptTemplate	.format()
Retriever	.get_relevant_documents()
Output parser	.parse()

4. The solution → Runnables

- A **Runnable** = a unit of work. Takes input → processes it → gives output.
- Every runnable follows the **same interface**:
  - **invoke()** → one input, one output
  - **batch()** → list of inputs at once
  - **stream()** → output comes in chunks
- Same interface ⇒ output of one = input of next. They connect like **Lego blocks**.
- A chain of runnables is **itself a runnable** → you can connect chains to chains.

input → Runnable A → Runnable B → Runnable C → output

5. "Nakli" LangChain (built from scratch in the video)

Idea: make an abstract **Runnable** class with one rule, "you must have **invoke()**". Every component inherits it, so a connector can just call **invoke** on each one in a loop.

```python
from abc import ABC, abstractmethod

class Runnable(ABC):                # the common interface
    @abstractmethod
    def invoke(self, input_data):
        pass

class NakliPromptTemplate(Runnable):
    def invoke(self, input_dict):
        return self.template.format(**input_dict)

class NakliLLM(Runnable):
    def invoke(self, prompt):
        return {'response': random.choice(self.responses)}

class NakliStrOutputParser(Runnable):
    def invoke(self, input_data):
        return input_data['response']

class RunnableConnector(Runnable):      # chain = also a Runnable!
    def __init__(self, runnable_list):
        self.runnable_list = runnable_list
    def invoke(self, input_data):
        for r in self.runnable_list:
            input_data = r.invoke(input_data)   # output → next input
        return input_data

chain1 = RunnableConnector([template1, llm, parser])
chain2 = RunnableConnector([template2, llm, parser])
final  = RunnableConnector([chain1, chain2])   # chain of chains ✓
```

Remember for exam ★

Runnables were created to **standardize** components. Common interface **(invoke / batch / stream)** → any component can connect to any other → no need for 100 different chain classes.

Video 9 – Runnables Part 2

Types of runnables + the primitives + LCEL

1. Two categories

Type	What it is	Examples
Task-specific	Core LangChain components converted into runnables. Each does **one specific task**.	**ChatOpenAI**, **PromptTemplate**, retrievers, **StrOutputParser**
Runnable primitives	Building blocks that **orchestrate** how other runnables run: in sequence, in parallel, or conditionally.	Sequence, Parallel, Passthrough, Lambda, Branch

2. RunnableSequence

Connects runnables one after another. Output of step 1 becomes input of step 2.




```
chain = RunnableSequence(prompt, model, parser)
chain.invoke({'topic': 'AI'})

# can keep going: joke → explain the joke
chain = RunnableSequence(prompt1, model, parser, prompt2, model, parser)
```

3. RunnableParallel

Runs many runnables at the same time. **Every branch gets the SAME input.** Output = a **dict** (one key per branch).



```
parallel_chain = RunnableParallel({
    'tweet':    RunnableSequence(prompt1, model, parser),
    'linkedin': RunnableSequence(prompt2, model, parser),
})
parallel_chain.invoke({'topic': 'AI'})
# → {'tweet': '...', 'linkedin': '...'}
```

4. RunnablePassthrough

Simplest one: returns the input exactly as it is, no change.

Why useful? Inside a parallel, to keep the original value next to a processed one. E.g. get the joke **and** its explanation in the final output.

```
parallel = RunnableParallel({
    'joke':        RunnablePassthrough(),              # joke as-is
    'explanation': RunnableSequence(prompt2, model, parser),
})
final = RunnableSequence(joke_gen_chain, parallel)
```

5. RunnableLambda

Turns any Python function into a runnable, so your own logic (cleaning, counting, formatting) can sit inside a chain.

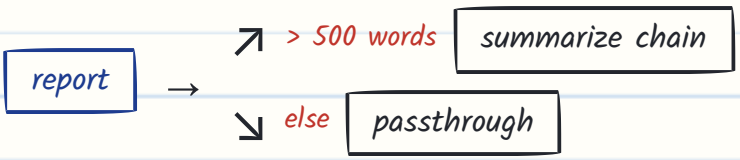
```
def word_count(text):
    return len(text.split())

parallel_chain = RunnableParallel({
    'joke':       RunnablePassthrough(),
    'word_count': RunnableLambda(word_count),   # or lambda x: len(x.split())
})
final_chain = RunnableSequence(joke_gen_chain, parallel_chain)
# → {'joke': '...', 'word_count': 23}
```

> **How did word_count get the text?** Sequence passed the joke string into the parallel → parallel gave that same string to every branch → **RunnableLambda** called **word_count(that_string)**. Lambda always gets the **whole input as ONE argument**.

6. RunnableBranch

The if / elif / else of LangChain. Checks conditions in order, runs the first one that's True, else runs the default.




```
# RunnableBranch( (condition, runnable), (condition, runnable), ..., default )
branch_chain = RunnableBranch(
    (lambda x: len(x.split()) > 500, RunnableSequence(prompt2, model, parser)),
    RunnablePassthrough()       # default (no tuple!)
)
final_chain = RunnableSequence(report_gen_chain, branch_chain)
```

BRACKET TRAP (my mistake!)

× **lambda x: len(x.split()>500, ...)** → **len(** swallows everything, no tuple formed

✓ **(lambda x: len(x.split()) > 500, chain)** → close **len()** right after **split()**, wrap condition + runnable in **( )**

Tip: define **is_long()** & **summarize_chain** first, then **(is_long, summarize_chain)**.

Note: **prompt2** got a plain string, not a dict, and it still worked → a PromptTemplate with only **one** variable auto-maps the string to that variable.

7. LCEL – LangChain Expression Language

RunnableSequence is used so often that LangChain gave a shortcut: the pipe **|**. Declarative, shorter, same result.

```
chain = RunnableSequence(prompt, model, parser)
# ⇕ exactly the same
chain = prompt | model | parser
```

- Works because every runnable supports **|** (it builds a sequence for you).
- Mix freely: **joke_gen_chain | parallel_chain**, **report_chain | branch_chain**.

~ ~ ~

Quick revision table

Primitive	Does	Output
Sequence	Runs steps one after another	last step's output
Parallel	Runs branches together on the same input	dict
Passthrough	Returns input unchanged	same as input
Lambda	Wraps a Python function	function's return
Branch	if/elif/else on conditions	chosen branch's output
LCEL |	Shorthand for Sequence	same as Sequence

Exam one-liners ★

- Task-specific = *does* the work · Primitives = *connect* the work
- Parallel → every branch, same input → dict
- Branch → **(cond, runnable)** tuples + default last
- **a | b | c = RunnableSequence(a, b, c)**